

Relatório de Análise de Log — Blue Team

***Nota sobre os dados:** o ambiente Docker não estava disponível para subir o lab e gerar um logs/access.log real a partir do ataque do Red Team. Para viabilizar a Fase 1 (Monitoramento), foi gerado um log sintético — access-log-simulado.log — no **mesmo formato** definido em proxy/nginx.conf / docs/observability.md, reproduzindo fielmente o cenário descrito no guia do Red Team: enumeração sequencial de /orders/:id, ativação do rate limiting e tentativa de bypass via rotação de IP. O script de análise (analise_log.py) é genérico e funciona igualmente sobre um access.log real — basta rodar python analise_log.py logs/access.log.*

Como reproduzir

```
python blueteam/evidence/analise_log.py blueteam/evidence/access-log-simulado.log
```

Saída completa em analise_output.txt.

1. IPs com mais requisições a /orders/

IP	Requisições	IDs distintos acessados	Classificação
203.0.113.77	600	600	Anômalo — enumeração sequencial
198.51.100.10	44	8	Tráfego legítimo (usuário 101)
198.51.100.45	42	8	Tráfego legítimo (usuário 104)
198.51.100.23	41	8	Tráfego legítimo (usuário 103)
203.0.113.78	25	25	Anômalo — bypass (rotação de IP)
203.0.113.79	25	25	Anômalo — bypass (rotação de IP)

Critério de anomalia usado: mais de 20 requisições/minuto de um único IP na rota /orders/ (usuários legítimos da API ficam bem abaixo disso — poucas requisições espaçadas em minutos). Os 3 IPs acima do limiar acessam um número de IDs distintos quase igual ao número de requisições — assinatura clássica de enumeração automatizada (BOLA), bem diferente do padrão dos usuários reais (sempre os mesmos 8 IDs, os seus próprios pedidos).

2. Distribuição de status HTTP

Status	Total	Significado no contexto
--------	-------	-------------------------

200	135	Pedidos retornados com sucesso (127 de tráfego legítimo + 8 indevidos , ids 1–8, todos os pedidos existentes)
404	379	IDs inexistentes (a API só tem 8 pedidos cadastrados — <code>api/src/data/seed.ts</code>) varridos pela enumeração
429	263	Bloqueios do rate limiting, após o hardening do Red Team (NGINX <code>limit_req_zone</code>)

Distribuição ao longo do tempo (ver linha do tempo abaixo): 100% dos 200/404 do atacante ocorrem antes do rate limiting; 100% dos 429 ocorrem depois.

3. Linha do tempo do ataque

Horário (UTC)	Evento
14:32:00	Primeira requisição anômala de 203.0.113.77 (GET <code>/orders/1</code>)
14:32:02	Volume cruza o limiar de anomalia (≥ 20 req/min) — padrão fica detectável
14:32:00 – 14:35:15	Enumeração sequencial ids 1–300, ~6–8 req/s, sem nenhum controle
14:35:15	Rate limiting (<code>limit_req_zone</code>) aplicado no NGINX
14:35:20 – 14:36:05	Red Team retoma o ataque (ids 301–600); primeiras respostas 429 aparecem
14:36:25 – 14:36:42	Tentativa de bypass com IPs novos (203.0.113.78, .79) e User-Agent rotativo — cada IP novo recebe sua própria cota (burst), depois é limitado também
14:36:52	Fim da atividade anômala

Duração total do incidente: $\approx$ 4 minutos e 52 segundos (14:32:00 → 14:36:52).

4. Tempo de detecção (MTTD)

Pergunta-guia do README: quanto tempo levou entre o início do ataque e o momento em que o log mostrou claramente o padrão anômalo?

- 1ª requisição anômala: 14:32:00
- Padrão estatisticamente claro (≥ 20 req/min do mesmo IP): 14:32:02
- **MTTD = ~2 segundos** para o padrão ficar visível nos dados — **mas isso só vale se houver monitoramento ativo/alertas em tempo real**. Sem isso, o MTTD real é "o tempo até alguém abrir o log manualmente", que pode ser horas ou dias. Essa distinção é o argumento central do Desafio Extra (proposta de alerta automático no PRI).

5. Pedidos e titulares expostos

- Pedidos indevidamente acessados (HTTP 200 vindos de IP anômalo): **ids 1 a 8 — ou seja, 100% da base de pedidos**.
- Titulares de dados afetados (usuários distintos donos desses pedidos, conforme `api/src/data/seed.ts`): **5 usuários** (101, 102, 103, 104, 105).

6. Desafio Extra — rate limiting reduziu o vazamento?

- **Antes do hardening:** 300 requisições em ~3min15s, sem nenhum bloqueio (0×429).
- **Depois do hardening:** das 325 requisições seguintes (entre o IP original e os dois IPs de bypass), 263 retornaram 429 — **~81% bloqueadas**.
- **Porém:** o rate limiting é por IP (`limit_req_zone ... $binary_remote_addr`). Os IPs 203.0.113.78 e .79 começaram cada um com sua própria cota de burst — ou seja, **distribuir a automação entre múltiplos IPs contorna parcialmente o controle**. Rate limiting por IP **mitiga**, mas não **corrige** a causa raiz (falta de verificação de propriedade no `GET /orders/:id`).
- **Critério de alerta proposto** (ver PRI, seção 7): disparar alerta quando um único IP exceder 20 requisições/minuto à rota `/orders/`, ou quando a razão (IDs distintos acessados ÷ requisições) de um IP for ≥ 0.9 — assinatura de varredura sequencial.

Ferramenta usada: Python (stdlib `json`, sem dependências externas) — ver `analise_log.py`.